

实验三

桌面物体自动分类整理

个人实验报告 · 杨牧青

场景与示教

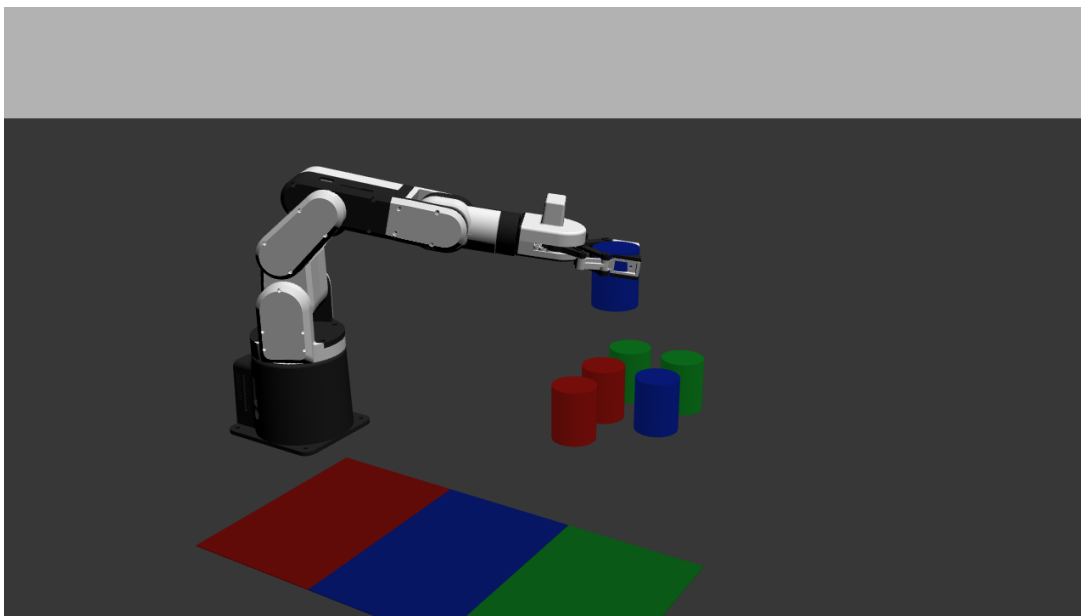
网格 · 分类垫 · 航点

动作执行

抓取 · 搬运 · 释放

结果核对

红蓝六目标 6/6



用于动作示教与分拣验证的 Gazebo 场景

姓名 杨牧青

学号 24020036076

课程 机器人学集成小组项目I · 第 17 组

项目仓库

<https://github.com/9161942-boop/robotics-experiment-3>

2026 年 9 月

1 实验基本要求

1.1 任务目标与考核重点

实验三的任务是让机械臂依据视觉结果完成桌面物体自动分类。程序从相机图像和检测数组中得到物体类别、置信度和位置，将检测框中心映射到固定取物网格，再依据类别选择对应的分类垫，依次完成接近、抓取、抬升、搬运、释放和结果记录。处理完一个目标后，任务节点继续处理后续目标。

任务验收包括自动决策、异常分支和证据完整性：运行时由系统自动确定网格和料盒；低置信度、网格外、重复占用和不可达目标生成可解释的跳过或停止结果；识别、抓取、放置和最终状态通过日志、画面和位姿记录复查。两类六物体的仿真基线要求至少正确整理五个目标。

1.2 实验环境与参数

本次个人工作在 Ubuntu 22.04、ROS 2 Humble 和 Gazebo Classic 11 环境中完成，使用 Python/rclpy 组织场景监视、任务规划和执行节点。机械臂为 mechArm 270 六自由度模型，夹爪通过 ros2_control 控制；俯视相机输出 640×480 图像，检测结果以 Detection2DArray 形式进入任务节点。

项目	最终采用的设置
场景	0.72 m×0.52 m 桌面；3 行 2 列六格取物区；红蓝绿三类圆柱
分类垫	红(0.14,-0.12)、蓝(0.26,-0.12)、绿(0.38,-0.12) m，横向排列
识别判断	置信度阈值 0.80；最近网格距离阈值 48 px
示教记录	manual_animation.json；27 个航点；34 步自动播放；每步 2.0 s
验收记录	任务 CSV、执行 CSV、异常日志、GUI 过程图和最终位姿

1.3 本次完成情况

我参与完成了仿真场景调整、手动示教、自动序列组织、抓取附着检查和结果核对。红蓝两类六个目标的对象级记录显示识别、抓取和放置均为 6/6，最终状态为 passed；红蓝绿三分类扩展完成了分类垫对齐、绿色目标航点和 34 步播放组织，现有归档材料包含场景和序列证据，三分类对象级成功率待独立 CSV 归档后再统计。

2 个人分工

我的工作集中在将检测和任务规划结果转化为稳定、可复查的机械臂动作，具体按“启动环境—手动示教—参数调试—自动播放—证据核对”的顺序展开。

- 检查 Gazebo 世界、控制器、相机和任务节点的启动关系，统一控制接口和进程实例。
- 使用手动控制台调节关节、夹爪和动作时间，记录每个网格的 above、pick、lift、release 航点。
- 核对物体几何尺寸、分类垫位置、置信度阈值和像素网格参数，使画面、规划和执行使用同一套坐标关系。
- 组织六个目标的自动播放子序列，检查 pick 和 release 航点的动作语义，并复核 manual_animation.json。
- 根据 attached、release、settled 和最终位姿等证据综合判定动作成功状态，建立可复查的验收依据。
- 使用小组统一提供的检测结果，核对网格编号、类别和任务队列在执行节点中的传递，并将核对结果与动作日志对应。

2.1 场景与启动组织

我先检查场景文件中的桌面尺寸、六个圆柱和三块分类垫是否全部在相机视野内，再确认世界坐标与任务节点使用的像素网格一致。手动调试时只启动世界、控制器和手动面板；自动验收时才启动任务播放器，并确保 Gazebo master、控制器和播放器各只有一个实例。

2.2 手动示教与航点记录

我使用关节滑块和夹爪参数逐点寻找安全高度、接近位置、抓取位置、抬升位置和释放位置。调试采用单变量方法，每次修改一个目标或一个动作阶段，确认末端与圆柱的相对位置后保存航点；航点记录同时标注接近、闭合、抬升和释放阶段，便于后续定位。

2.3 参数与抓取验证

我重点检查圆柱是否位于两指之间、夹爪闭合后目标是否随末端抬升，以及释放前后 X/Y 是否保持一致。对于仿真中的逻辑附着，我把“目标高度超过 0.075 m”作为画面取证条件，并同时检查 attached 事件，用两项证据区分夹爪闭合和成功抓取。

2.4 自动序列与日志整理

我先运行单个目标，再运行一个目标的完整子序列，最后才组合六个目标。航点名中包含 pick 的位置触发抓取，包含 release 的位置触发释放，其余位置只表示运动。自动运行后，我对照对象编号、网格编号、类别、事件和最终位姿，整理出可与截图相互印证的 CSV 和异常记录。

2.5 集成验收

最后我用红蓝六目标场景进行对象级基线检查，再单独检查红蓝绿扩展场景的分类垫排列和自动播放顺序。扩展结果按场景证据、接口工作和对象级数据分别归档，红蓝基线的 6/6 结果单独列示。

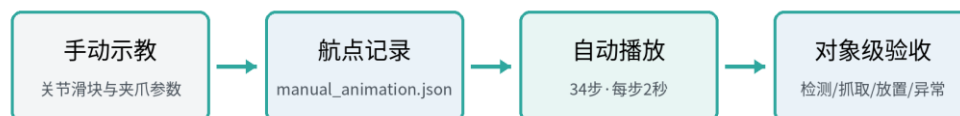


图 1 我的工作手动示教到对象级验收链路中的位置

3 项目实现原理

3.1 系统组成与数据流

从个人调试角度看，系统可以分为场景、感知、规划、执行和证据五层。Gazebo 提供桌面、圆柱、分类垫和机械臂模型；监视节点发布 Image 和 Detection2DArray；任务节点完成置信度筛选、网格匹配、类别查询和队列去重；执行节点读取队列并完成抓取和释放；GUI、事件日志、CSV 和最终位姿共同组成验收证据。



图 2 仿真系统中识别、网格定位、分类和执行模块的连接

3.2 网格定位与分类决策

我参与核对的核心关系是“检测框中心—最近网格—类别—分类垫”。任务节点把检测框中心投影到图像坐标，并与 3 行 2 列的固定格点比较；最近格点距离不超过 48 px 才生成 grid_id。类别 red、blue、green 通过显式配置表分别指向 red_sort_bin、blue_sort_bin 和 green_sort_bin。未知类别、低置信度结果和同一网格重复检测被标记为 skipped，并生成对应原因。

红蓝两类定量基线的六个占用网格

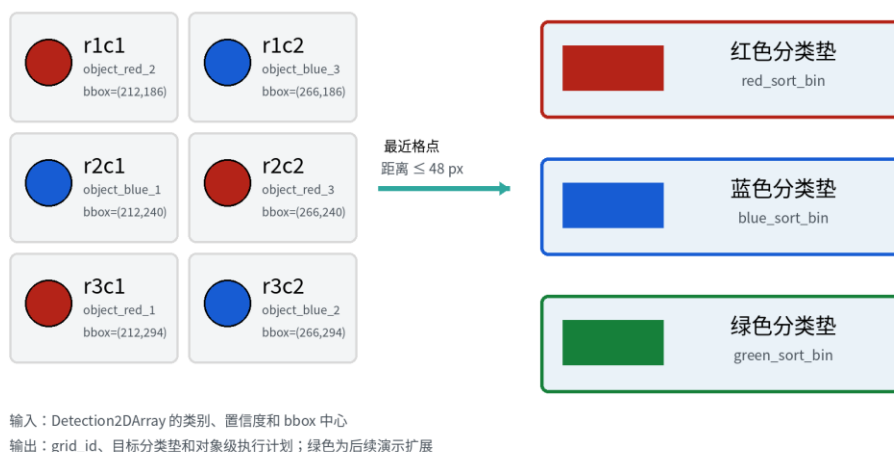


图 3 我参与核对的六个网格、类别和分类垫关系

3.3 抓取与搬运动作链

一个可复现的抓取被拆成六个阶段：先到安全高度 above，再下降到 pick，闭合夹爪后检查末端与目标距离，确认附着后 lift，最后搬运到分类垫上方。搬运过程中保存并更新目标相对夹爪的偏移，使目标持续跟随末端运动。

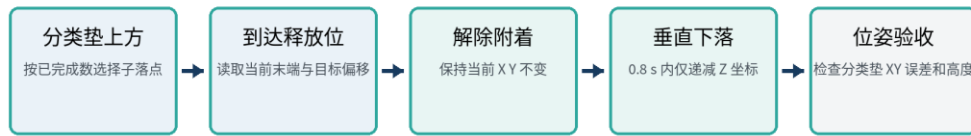


抓取成功判据：夹爪已闭合，目标位于两指之间，且目标 z 坐标高于桌面静止高度。
失败处理：超过位置阈值则拒绝附着；服务不可用则记录异常，不宣告抓取成功。

图 4 我调试的接近、闭合、检查、附着和抬升动作

3.4 垂直释放与落点检查

释放阶段读取当前末端位置，保持 X/Y 对齐并沿 Z 方向下降；解除附着后等待目标稳定，再读取最终位姿。对于同一颜色的多个目标，为它们分配不同落点，形成清晰的放置顺序。



为什么不写死坐标：释放起点由当前实际夹爪位姿决定，避免目标瞬间横移到分类垫中心。

同色不重合：同一分类垫为第 1 个和第 2 个目标预留不同子位置。

图 5 垂直解除附着和落点检查流程

3.5 航点语义与状态证据

manual_animation.json 保存已经验证过的航点和动作顺序，任务节点负责类别决策。播放器根据包含 pick、release 的航点名触发相应语义，其余航点承担运动。一次任务通过的条件包括 GUI 几何关系、状态反馈、attached/release 事件和最终位姿检查，四类证据共同确认结果。



图 6 手动控制台中的关节、夹爪和航点列表



图7 六个目标的自动播放序列

4 项目中遇到的问题和解决方法

本节把调试过程按问题逐项复盘。每项都记录实际看到的画面或日志、对应的检查位置、具体修改的航点、参数或节点，以及修改后的验证条件。红蓝基线的对象级结论以mecharm_sorting_execution.csv为准，三分类扩展只写已经有场景和序列证据的内容。

4.1 默认姿态导致夹爪偏离目标

现象：三分类场景第一次播放绿色目标时，机械臂从 above_green_1 下到 pick_4 后，夹爪中心没有落在圆柱中心线上，右侧指尖先碰到圆柱边缘；above_green_2/pick_5 也出现相同趋势。画面中夹爪虽然闭合，但圆柱没有被稳定夹在两指之间。

原因分析：我在 manual_control_panel 中暂停在上述两个航点，逐个拖动关节滑块，对照 sorting_scene.yaml 中六个取物点和俯视图观察末端位置。原来的默认姿态是为固定点 A-B 标定的，直接复用到绿色格点时，末端接近方向与圆柱中心不一致；因此问题不在夹爪开度，而在每个网格对应的 above/pick 姿态没有单独校准。

处理方法：以红蓝基线已经验证的动作顺序为模板，重新保存 above_green_1→pick_4 和 above_green_2→pick_5 两组航点：above 阶段保持夹爪开度 0.15、接触垫力 -4.0，pick 阶段切换到夹爪 -0.31，并把每个动作时长固定为 2.0 s。每次只修改一组航点，先在 GUI 中单步执行，再写回 manual_animation.json，最后重新组织 34 步总序列。

验证结果：单目标测试时，两指从圆柱两侧同时接近，夹爪闭合后没有明显侧滑；随后按 manual_animation.json 的六目标顺序播放，绿色目标的 pick_4/pick_5 不再与边缘碰撞。验证依据是手动控制台截图、六目标序列图和 Gazebo 录屏，不把这部分扩写成三分类对象级成功率。

4.2 夹爪闭合但物体没有离桌

现象：一次蓝色圆柱测试中，日志已经出现 pick 和 close_gripper，但圆柱仍停在桌面约 $z=0.0275$ m，机械臂抬升后夹爪下方为空；仅凭夹爪闭合会把这次动作误判为成功。

原因分析：我把动作拆成闭合、距离检查和抬升三段，发现原来的播放器只发布夹爪和机械臂的 JointTrajectory，没有保存目标与末端的相对位姿。修复前，sorting_executor.py 没有把 /gazebo/model_states 中的目标位姿和 /gazebo/link_states 中的夹爪位姿绑定起来，所以“命令已经发出”不等于“目标已经附着”。

处理方法：在 sorting_executor.py 中加入抓取对象选择和距离检查，抓取位置容差设为 0.05 m；确认对象后保存 attach_offset，并通过 /gazebo/set_entity_state 以约 0.06 s

的周期更新被选模型的位置。状态机同时写入 `grasp_attached` 事件；未找到近邻目标时进入 `failed/exception` 分支，并把目标高度超过 0.075 m 作为画面取证条件。

验证结果：重新运行单目标测试后，`blue_cylinder_grasp_verified.png` 中可以看到圆柱位于两指之间并已离开桌面。对应 CSV 依次出现 `grasp_attached`、`lift`、`release` 和 `object_complete` 事件，状态均为 `passed`，说明验证的是目标确实随末端抬升，而不是只看夹爪是否闭合。

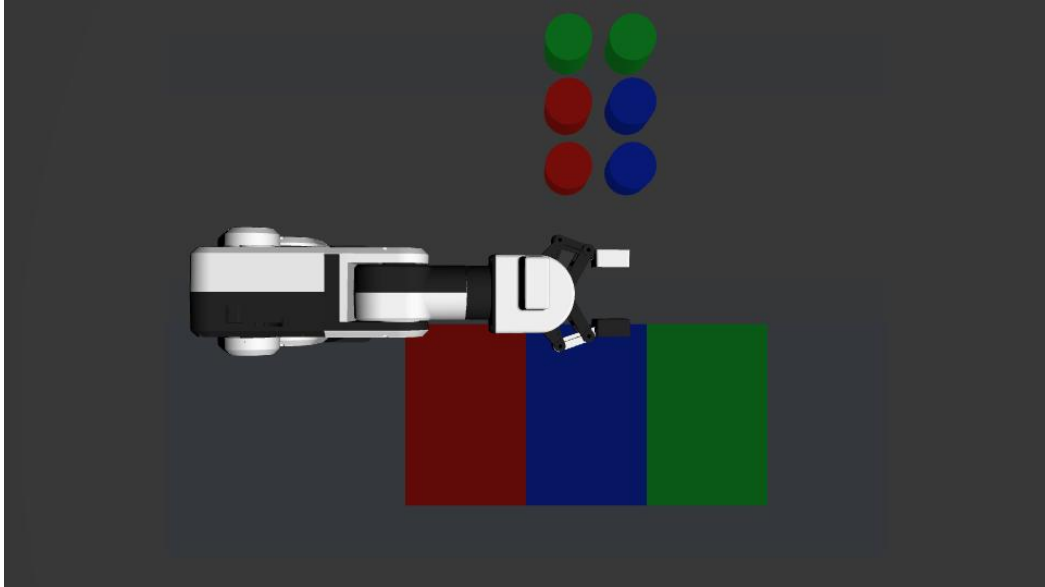


图 8 夹爪闭合、目标位于两指之间且已离开桌面的验证画面

4.3 释放跳变和同色目标重叠

现象：第一次释放测试把目标直接写到分类垫中心，画面没有连续的对准、解除附着和下落过程；同一颜色的两个目标还会落在完全相同的位置，后一目标遮住前一目标，无法从最终画面判断是否逐个完成。

原因分析：检查 `release` 阶段后，我发现一次 `SetEntityState` 调用同时改变了目标的 X、Y、Z，目标会在相邻帧之间横向跳变，原来的播放序列也没有为同色目标维护不同的落点索引。因此即使最终坐标看起来正确，释放过程仍不符合验收时对动作连续性和结果可复查性的要求。

处理方法：在执行器中为 `red`、`blue`、`green` 分别建立落点列表：红色使用 (0.115,-0.15)、(0.14,-0.12)、(0.165,-0.09)，蓝色使用 (0.235,-0.15)、(0.26,-0.12)、(0.285,-0.09)，绿色使用 (0.34,-0.02)、(0.38,0.02)。释放时保持 X/Y 对齐，解除附着后再沿 Z 方向下落，并在放开夹爪后等待稳定；验证阶段检查 XY 误差不超过 0.07 m，超出则记录 `release pose outside destination`。

验证结果：`release_stages.png` 中连续呈现了到达释放位、解除附着、垂直下落和稳定四个阶段，同色圆柱的最终落点也形成间隔。CSV 的 `release` 记录保存了 `destination` 坐标，随后写入 `object_complete`；因此这一修改同时解决了画面跳变和同色目标重叠两个问题。

4.4 世界坐标和像素网格不同步

现象：调整场景坐标后，任务节点只识别到少数目标，完整计划中出现空缺；从俯视图看六个圆柱都在桌面上，但任务日志中的目标数量与画面不一致。这个问题导致后续抓取序列缺少对象，并不是机械臂动作本身失败。

原因分析：我对比世界中三行目标的位置、`sorting_scene_monitor.py` 的投影公式和 `sorting_task.py` 使用的像素网格，发现场景已经变化而旧网格参数仍被加载。当前投影把 `world-y` 映射到图像横坐标、`world-x` 映射到图像纵坐标，若仍使用旧行坐标，检测框中心就会被分到错误网格或判定为网格外。

处理方法：根据新的投影关系重新计算三行像素坐标 186、240、294，并保留列坐标 212、266、320；同时把 `grid_tolerance_px` 固定为 48，清理旧实例后重新启动 `monitor`、`task` 和 `executor`。为避免目标搬走后再次触发空计划，还保留了检测签名去重和 `execute_motion` 下的 `plan_locked` 保护。

验证结果：重新启动后，`grid_mapping.png` 中的检测框中心与六个网格一一对应，日志中的 `grid_id` 不再重复或缺失。红蓝基线 CSV 的 `task_result` 记录 `count=6`、`success_count=6`，六个对象分别得到唯一网格，说明坐标修正已经传递到任务队列和最终结果。

4.5 重复启动导致动作不发送

现象：界面看起来正常，但播放器发送动作后机械臂没有响应，或者同一动作被重复触发；在终端中可以看到同一阶段的输出出现两次。此时如果继续调节航点，会把进程冲突误认为姿态参数错误。

原因分析：我检查 `world`、`controller` 和播放器进程，发现旧 Gazebo master 与新实例同时存在，手动控制台也可能连接到旧的播放器。多个实例会同时监听控制主题，导致一套命令被重复消费或发往已经失效的控制器。

处理方法：先结束旧的 `gzserver`、`gzclient` 和播放器进程，只保留一个世界实例，再按 `start_contact_grasp.sh` 的顺序启动 Gazebo、控制器和 `manual_control_panel`。自动模式只启动一个 `sorting_gazebo.launch.py`，并利用任务节点的检测签名去重和 `plan_locked` 标志，避免同一份计划被重新送入执行器。

验证结果：清理环境后，控制器只收到一套动作命令；单目标测试能够从 `above`、`pick`、`lift` 推进到 `release`，34 步完整序列也不再重复触发。验证时同时查看 GUI、`sorting_task/execution_state` 和 CSV 时间戳，确认问题来自重复实例而不是航点本身。

4.6 节点启动日志报错

现象：任务节点启动后立即退出，终端提示日志调用参数不兼容，因而没有进入 `detecting` 或 `plan_ready` 状态。此时 Gazebo 画面仍然存在，但没有任务状态和对象级结果，后续抓取不能作为有效实验记录。

原因分析：我把错误定位到 Humble Python logger 的调用方式：旧写法把额外参数交给日志函数处理，异常发生在真正进入任务状态机之前。通过查看节点退出位置和启动后的状态主题，确认这不是检测框或机械臂控制问题，而是节点初始化阶段的日志调用错误。

处理方法：将相关日志改为兼容 Humble 的 f-string 写法，重新构建工作空间，并在干净终端中重新启动统一 Launch。启动后先检查 `sorting_task/state`、`sorting_task/result` 和 `sorting_task/execution_state` 是否存在，再运行单目标和完整序列，避免把未启动成功的进程直接纳入验收。

验证结果：修复后节点保持运行并持续发布 `detecting`、`plan_ready`、执行阶段和 `task_passed` 状态，CSV 也能记录 `plan`、`grasp_attached`、`release` 和 `task_result`。这样抓取问题可以在稳定的启动环境中单独分析，日志错误不再掩盖任务本身的执行结果。

问题	我的判断依据	修复后的检查
姿态不准	末端与目标的相对位置	抓取距离与对准画面
未附着	目标高度和 <code>attached</code> 事件	目标随夹爪抬升
释放异常	释放前后 X/Y、Z 变化	垂直下落和分离落点
网格错误	投影像素与 <code>grid_id</code>	六个目标唯一编号
运行冲突	<code>world/controller</code> 数量	单实例播放

问题	我的判断依据	修复后的检查
节点退出	终端错误位置	节点持续发布状态

5 仿真结果与个人总结

5.1 红蓝基线对象级结果

红蓝两类六个目标的对象级 CSV 是本次量化验收的主要依据。六个目标的识别置信度均为 0.99，抓取和放置结果均完成，最终状态为 passed，success_count=6。这个结果满足任务书“至少正确整理 5 个物体”的红蓝基线要求。

类别	对象数	识别	抓取	放置	结论
蓝色圆柱	3	0.99	3/3	3/3	通过
红色圆柱	3	0.99	3/3	3/3	通过
合计	6	6/6	6/6	6/6	通过

5.2 三分类扩展结果的边界

我参与完成了红蓝绿三分类场景、分类垫对齐和自动播放序列。扩展阶段证明新增绿色类别可以复用检测消息、网格定位、类别映射和执行接口；当前归档材料包含场景、航点和播放证据，三分类对象级 CSV 完成归档后再统计对应成功率。



图 9 三分类扩展中识别、定位、分类和执行模块的连接

5.3 证据链与调试工具

GUI

用来检查末端、夹爪和目标的几何关系；状态反馈用来判断当前处于扫描、规划、执行还是释放阶段；事件日志用来确认 attached、release 和 settled 是否按顺序发生；最终位姿用来判断落点；对象级 CSV 则把这些观察按 object_id 汇总。多层证据互相印证后，才能区分“动作完成”和“任务成功”。



验收原则：单独的动画或单条“完成”日志都不足以证明整理正确，必须将过程、事件和最终落点互相印证。

图 10 我整理的过程画面、状态、事件和 CSV 证据链

5.4 个人总结

这次实验让我认识到，机械臂仿真要求同时保证关节运动、场景模型、控制接口、任务逻辑和验收证据的一致性。一个动作只有在“目标被正确选中、夹爪与目标相对位置满足条件、目标高度发生变化、事件顺序完整、最终位姿落在分类区域”时，才具备可复查的成功依据。这个认识改变了我对仿真结果的判断方式：我开始把画面、状态和数据看成同一件事的不同观测，而不是只看机械臂是否完成了一段轨迹。

在调试过程中，我逐渐形成了固定的排查顺序。第一步是把问题缩小到扫描、网格定位、接近、抓取、搬运、释放或验收中的一个阶段；第二步是用单目标和手动面板复现现象，记录末端位置、目标位置、夹爪状态和当前航点；第三步是一次只修改一个变量，例如像素网格、抓取高度、夹爪开度、释放高度或动作时间；第四步是用事件日志、最终坐标和完整序列复核修改结果。网格错位、释放跳变、目标未附着和重复启动等问题，都按照这套顺序得到定位。

- 用阶段化思路分析动作：先确认任务进入了哪个状态，再检查该状态的输入、输出和退出条件。
- 用几何关系验证抓取：同时观察指垫中心、圆柱中心、目标高度和相对位姿，确定运动轨迹与对象状态一致。
- 用最小实验减少干扰：先单点、再单目标子序列、最后整序列，降低多个问题同时出现时的判别难度。
- 用对象级记录闭环：把 object_id、grid_id、类别、事件、最终位姿和异常码对应起来，使每个目标都能单独复核。
- 用干净环境复现：控制 Gazebo world、控制器和播放器实例数量，保证每轮测试的启动条件相同。

这次实验还让我熟悉了模块之间的接口责任。任务节点负责生成对象顺序和分类目标，执行节点负责完成动作与反馈，场景和日志负责提供复核依据。我的工作位于任务规划结果与仿真验收之间，需要把抽象的任务项落实成可执行的姿态、航点和状态检查。这使我更加重视命名、参数记录、日志字段和图片说明的完整性，因为它们直接影响小组成员之间的协作和后续复现。

仿真中的逻辑附着提升了动作重复性，也让我看清了仿真与真机之间的差异。真实系统还需要重新标定相机映射、夹爪摩擦、接近高度、闭合速度、释放等待和急停距离。后续迁移到真机时，我会沿用“低速单目标—检查识别—检查抓取—检查释放—加入第二类—扩展到六目标”的顺序，每一步保存对象级记录，再决定是否进入下一阶段。

6 结论

我完成了实验三仿真执行侧的主要个人工作，并把个人贡献落实到场景、航点、状态和证据四个层面。通过场景调整、手动示教、动作修复、自动播放和对象级复核，我参与建立了从任务规划结果到机械臂动作、再到实验结论的完整链路。红蓝六个目标的对象级结果达到 6/6，三分类扩展完成了场景、分类垫和自动序列的联调。在执行侧，我使用小组提供的检测结果检查后续网格映射、抓取和释放流程。

6.1 个人工作完成情况

工作模块	完成内容	对应结果
场景与启动	调整六格桌面、红蓝绿圆柱和三块分类垫；检查 world、控制器、相机和播放器的启动关系	三分类场景稳定加载，单实例运行
航点与动作	建立 above、pick、lift、release 等语义航点，组织六目标子序列和统一动作节拍	manual_animation.json 保存 27 个航点，自动播放 34 步
抓取与释放	校准末端参考点，加入距离检查、目标跟随、垂直释放和同色不同落点	目标能够离桌、跟随搬运并落入对应分类垫
问题修复	处理姿态偏差、网格不同步、重复启动、节点日志报错和释放跳变	单目标与整序列复跑通过
结果核对	对照 GUI、状态、事件、最终位姿和对象级 CSV 复核结果	红蓝基线对象级结果为 6/6

6.2 个人能力收获

我掌握了 Gazebo 场景参数与 ROS 2

控制器之间的联调方法，能够从世界坐标、像素网格、末端姿态和对象状态四个角度检查同一动作。通过手动控制台和 JSON

航点文件，我熟悉了从单点测试到完整序列的动作组织方式；通过状态主题、事件日志和对象级 CSV，我能够把一次运行拆分为可定位、可比较的阶段。

在协作方面，我进一步理解了统一接口和统一命名的重要性。任务节点输出的 object_id、grid_id 和 target_bin，必须与执行节点的航点、日志和最终位姿相互对应，报告中的图片和表格也要能够回到具体对象。这样的记录方式提高了问题定位效率，也使个人调试结果能够直接支持小组验收。

6.3 后续改进计划

后续真机工作将先完成相机单应性标定和桌面网格标记，再用单个目标验证识别、PICK、抬升、PLAC E 和回

HOME。确认抓取高度、夹爪开度、释放等待和急停策略后，再加入第二类目标并扩展到六个目标。每个阶段继续保存真实类别、预测类别、置信度、网格编号、抓取结果、放置结果和异常码，便于与当前仿真字段对照。

从个人方法上，我会继续保持“阶段定位、单变量修改、最小案例复现、完整数据复核”的工作习惯，并补充运行 ID、日志轮转、连续帧稳定判定和异常恢复记录，使后续实验具备更好的可重复性和安全性。

6.4 GitHub同步记录

我参与整理的实验三工程、证据和报告已同步到 GitHub 仓库：<https://github.com/9161942-boop/robotics-experiment-3>。

仓库中保留 ROS 2 工程、Gazebo 场景、manual_animation.json、对象级执行记录、过程截图以及小组和个人报告；两段 Gazebo 仿真演示视频也已放入 videos/ 目录。提交历史截图和说明用于证明材料已经同步到公共仓库，个人报告中的问题、调整和验证记录均可在报告、截图、日志及仓库目录之间对应。

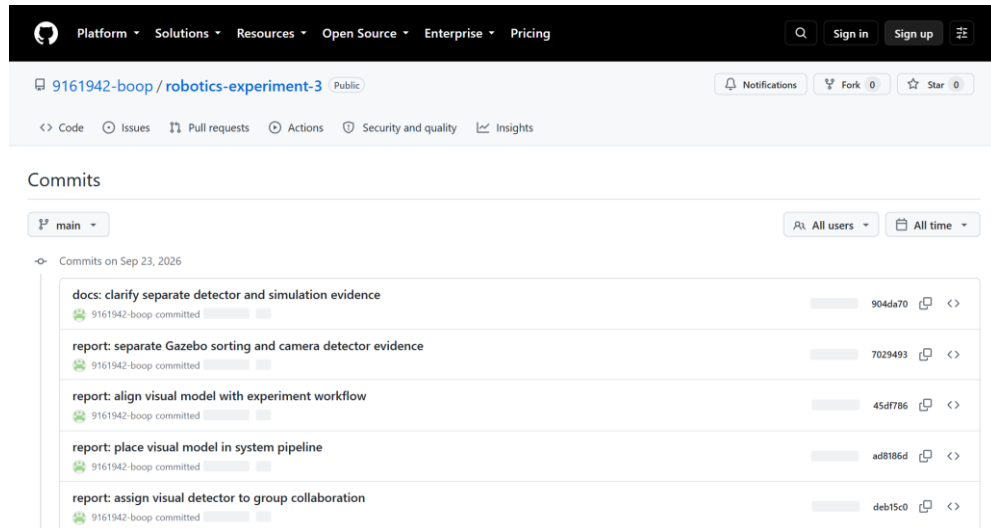


图 11 个人实验报告对应的 GitHub 实验三仓库提交历史

个人工作对应的仓库证据

个人工作	对应文件	核对要点
场景与网格	code/src/mecharm_fixed_pick_sim/worlds/sorting_desktop.world	对照六个取物格、圆柱及分类垫位置与场景图。
航点组织	code/manual_animation.json	核对 above、pick、lift、release 航点和六目标播放顺序。
问题调试	evidence/ERROR_LOG.md	对照网格偏移、附着失败和释放跳变的处理记录。
结果复核	evidence/mecharm_sorting_execution.csv	按 object_id 核对分类区、抓取释放事件和六目标终态。